

Assignment 3A Global Baseline Estimate LLM Transcript – Brandon Chanderban

The following outlines the interactions I had with the LLM during the completion of this assignment, arranged in the order they occurred. Questions are denoted in green.

Q: How do I convert a data table (raw_ratings) from a wide format to a long one? It currently has rows (Critics) corresponding to users/reviewers, columns corresponding to movies, and ratings at the intersections. I also want to rename the Critic label to user.

To convert your raw_ratings dataframe from **wide** format to **long** format in R (using tidyverse), you would use the pivot_longer() function from **tidyr**.

◆ Your Current Structure (Wide Format)

- Rows → Critics (users)
- Columns → Movie titles
- Cells → Ratings (with some NAs for missing ratings)

You want to restructure it into:

user | movie | rating

✓ Step-by-Step Code

```
long_ratings <- raw_ratings %>%  
  pivot_longer(  
    cols = -Critic,    # Keep Critic column fixed  
    names_to = "movie", # New column for movie titles  
    values_to = "rating" # New column for ratings  
  ) %>%  
  rename(user = Critic) # Rename Critic column to user
```

🔍 What Each Part Is Doing

1 pivot_longer()

```
cols = -Critic
```

This tells R:

“Take every column **except** Critic and pivot them.”

So all movie columns become rows.

```
names_to = "movie"
```

The **column names** get stacked into a new column called "movie".

```
values_to = "rating"
```

The actual numeric ratings get stacked into a new column called "rating".

2 rename(user = Critic)

This simply changes the column name from "Critic" to "user" so it aligns with the structure of your recommender system logic.

If you'd like, I can also explain **why long format is required for grouped operations and joins**.

Q: I have stored the global mean for my dataset into a variable mu, can you set out the R code required to garner the movie bias from my long_ratings dataframe?

Since you've already stored the **global mean** in a variable called mu, the movie bias (b_i) is computed as:

$$b_i = \text{movie average} - \mu$$

Here is the exact R code using your long_ratings dataframe:

```
movie_bias <- long_ratings %>%  
  group_by(movie) %>%  
  summarise(  
    movie_avg = mean(rating, na.rm = TRUE),  
    b_i = movie_avg - mu  
  )
```

What This Code Is Doing (Step-by-Step)

1 group_by(movie)

This groups the dataset by each individual movie so that calculations happen **within each movie**, not across the entire dataset.

2 summarise()

```
movie_avg = mean(rating, na.rm = TRUE)
```

This calculates the **average rating for each movie**, excluding missing values.

```
b_i = movie_avg - mu
```

This calculates the **movie bias**:

- If positive → movie is rated above average
 - If negative → movie is rated below average
-

Q: I calculated the user bias using the same approach as directed for the creation of the movie bias dataframe. It is called user_bias. How do I go about using the dataframes to make the prediction of a user-movie pairing? For instance, if the user is called Matt and I want to predict his rating for Deadpool?

Perfect — since you now have:

- `mu` → global mean
- `user_bias` → contains user and `b_u`
- `movie_bias` → contains movie and `b_i`

You're ready to compute:

$$\hat{r}_{ui} = \mu + b_u + b_i$$

 Method 1 (Clean + Reproducible Way Using `tibble()` + `left_join()`)

This mirrors exactly what you implemented in your assignment.

```
prediction <- tibble(
  user = "Matt",
  movie = "Deadpool"
) %>%
  left_join(user_bias, by = "user") %>%
  left_join(movie_bias, by = "movie") %>%
  mutate(predicted_rating = round(mu + b_u + b_i, 2))
```

prediction

 What This Is Doing

1 Create a mini table

```
tibble(user = "Matt", movie = "Deadpool")
```

This creates a one-row dataframe representing the pairing you want to predict.

2 Join in Matt's user bias

```
left_join(user_bias, by = "user")
```

This attaches Matt's `b_u`.

3 Join in Deadpool's movie bias

```
left_join(movie_bias, by = "movie")
```

This attaches Deadpool's `b_i`.

4 Compute prediction

```
mutate(predicted_rating = round(mu + b_u + b_i, 2))
```

Applies the Global Baseline formula.

✓ Method 2 (Direct Pull Method — More Manual)

If you just want the number:

```
user_b <- user_bias %>%  
  filter(user == "Matt") %>%  
  pull(b_u)
```

```
movie_b <- movie_bias %>%  
  filter(movie == "Deadpool") %>%  
  pull(b_i)
```

```
round(mu + user_b + movie_b, 2)
```

🎯 Why Method 1 Is Better for Your Assignment

- It scales.
 - It works for generating all missing predictions.
 - It mirrors how recommender systems operate structurally.
 - It reads clearly in a report.
-

If you'd like, I can now show you how to automatically rank all of Matt's unseen movies from highest to lowest predicted rating.

Q: How do I do a lump sum prediction for all of the unrated movies within the dataset, and then rank them in descending order of predicted_rating per user?

Excellent — this is exactly where the Global Baseline approach becomes powerful.

You already have:

- mu
- user_bias
- movie_bias
- long_ratings (with user, movie, rating)

Now we'll:

1. Compute predictions for **all user–movie pairs**
 2. Filter to only **unrated movies**
 3. Rank them **descending per user**
-

✓ Step 1 — Generate Predictions for Every Pair

```
all_predictions <- long_ratings %>%  
  left_join(user_bias, by = "user") %>%
```

```
left_join(movie_bias, by = "movie") %>%
mutate(predicted_rating = round(mu + b_u + b_i, 2))
```

What this does:

- Joins in user bias (b_u)
- Joins in movie bias (b_i)
- Applies:

$$\hat{r}_{ui} = \mu + b_u + b_i$$

Now every row has a predicted rating — even ones that already have actual ratings.

✅ **Step 2 — Keep Only the Missing Ratings**

```
recommendations <- all_predictions %>%
  filter(is.na(rating))
```

This ensures we only recommend movies the user **has not rated yet**.

✅ **Step 3 — Rank Descending Per User**

```
recommendations_ranked <- recommendations %>%
  group_by(user) %>%
  arrange(desc(predicted_rating), .by_group = TRUE)
```

Why .by_group = TRUE matters:

It ensures the ranking happens **within each user**, not across the entire dataset.

✅ **Final Output**

```
recommendations_ranked
```

You will now see:

- Each user
- Their unrated movies
- Ordered from highest predicted rating to lowest

Q: Can you assist with creating a predictor function of sorts? It needs the functionality of taking in a user and movie name/pair as the argument, and then automatically calculates the estimated movie rating. It should have a way to deal with misspelt names and titles.

This is a clean implementation of a predictor function, using the Global Baseline formula.

```
predict_rating <- function(desired_user, desired_movie){

  user_b <- user_bias %>%
    filter(user == desired_user) %>%
```

```

pull(b_u)

movie_b <- movie_bias %>%
  filter(movie == desired_movie) %>%
  pull(b_i)

if(length(user_b) == 0 | length(movie_b) == 0){
  return("User or movie not found.")
}

round(mu + user_b + movie_b, 2)

}

```

What This Version Does

1 Extract User Bias

It looks inside the `user_bias` dataframe and finds the `b_u` value corresponding to the provided user.

If the user exists, it pulls the numeric bias value.

If not, `user_b` becomes a zero-length vector.

2 Extract Movie Bias

It does the same process with the `movie_bias` dataframe to retrieve `b_i`.

3 Basic Error Handling

This line:

```
if(length(user_b) == 0 | length(movie_b) == 0)
```

Checks whether either:

- The user does not exist
- The movie does not exist

If either fails, it returns:

"User or movie not found."

This is the simplest form of handling misspelled names or incorrect titles.

4 Compute the Prediction

If both exist, it calculates:

$$\hat{r}_{ui} = \mu + b_u + b_i$$

And rounds the result to two decimal places.

Q: I've tested the function, but I would have noticed that it also provides a predicted rating, even for those movie-user pairs with actual ratings available. In this, could you tailor the function so that in the event of an existing move-rating pair, it outputs a string saying that the user A would have seen movie Y before. If not, output the predicted value as per normal.

Yes — this is exactly the right refinement to make.

Right now, your function always computes:

$$\hat{r}_{ui} = \mu + b_u + b_i$$

even when an **actual observed rating already exists**, which defeats part of the recommender logic.

We can fix this by:

1. First checking whether a real rating exists in long_ratings
2. If it does → return a message with the observed rating
3. If it does not → compute the predicted rating as before

Here is the revised function:

```
predict_rating <- function(desired_user, desired_movie){
```

```
  # Step 1: Check if rating already exists
```

```
  existing_rating <- long_ratings %>%
```

```
    filter(user == desired_user,  
           movie == desired_movie,  
           !is.na(rating)) %>%  
    pull(rating)
```

```
  # If user already rated movie
```

```
  if(length(existing_rating) > 0){  
    return(paste0(  
      desired_user,  
      " has viewed this movie before. A rating of ",  
      existing_rating,  
      " was given."  
    ))  
  }
```

```
  # Step 2: Otherwise compute baseline prediction
```

```
  user_b <- user_bias %>%  
  filter(user == desired_user) %>%
```

```

pull(b_u)

movie_b <- movie_bias %>%
  filter(movie == desired_movie) %>%
  pull(b_i)

if(length(user_b) == 0 | length(movie_b) == 0){
  return("User or movie not found.")
}

predicted <- round(mu + user_b + movie_b, 2)

return(paste0(
  "Predicted rating for ",
  desired_user,
  " on ",
  desired_movie,
  " is ",
  predicted
))
}

```

What Changed

New Step Added (Very Important)

```
!is.na(rating)
```

This ensures that only *actual observed ratings* are captured — not missing values.

Then:

```
if(length(existing_rating) > 0)
```

If a rating exists, we stop immediately and return a message instead of computing a prediction.

Behavior Summary

Scenario	Output
User already rated movie	Message with actual rating
Movie unseen by user	Predicted rating
Misspelled user/movie	"User or movie not found."

This now behaves like a proper recommender interface — it does not overwrite known information with a prediction.